End-to-End WireGuard VPN Deployment & Testing Guide

Deployment Path: Windows Laptop Client (India) → WireGuard Encrypted Tunnel → Azure Ubuntu VM (East US) → Public Internet

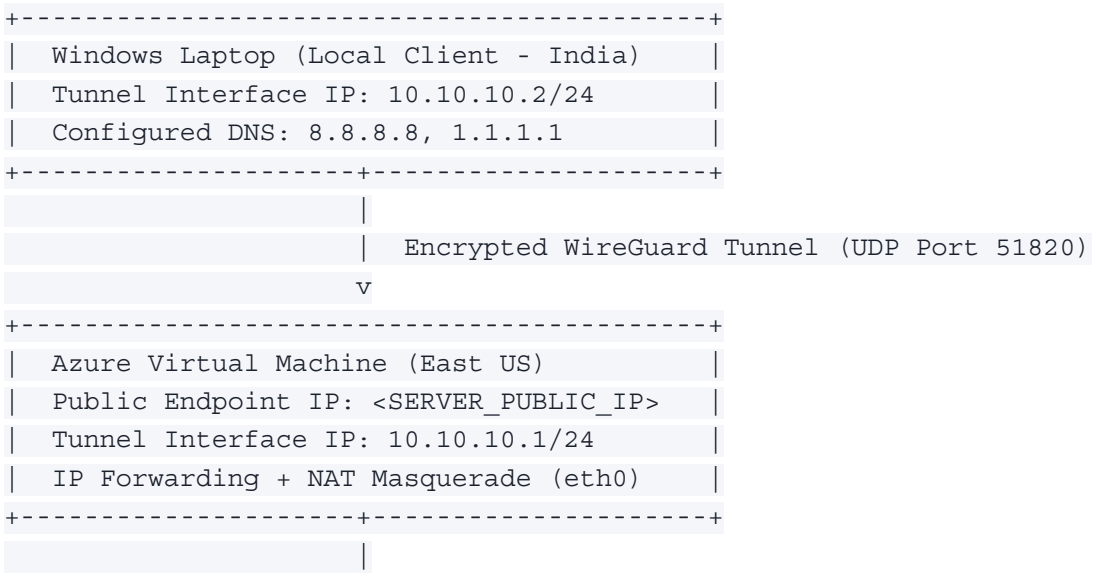
This guide provides a complete walkthrough for deploying a self-hosted **WireGuard VPN** server on Microsoft Azure (East US), routing all traffic from a Windows laptop client through the encrypted tunnel, and measuring real-world latency, throughput, and application load times.

Important Notice

> This project is intended as a **Proof of Concept (PoC)** and learning exercise demonstrating how to deploy, validate, and benchmark a self-hosted WireGuard VPN on Microsoft Azure. While functional for testing and personal use, production deployments should incorporate additional controls such as centralized logging, monitoring, backup procedures, access controls, patch management, key rotation, and high-availability architecture.

1. System Architecture & Prerequisites

WireGuard operates inside the Linux kernel using modern cryptography (ChaCha20-Poly1305), delivering lower latency and higher throughput than legacy protocols such as OpenVPN or IPsec. All client IPv4 and IPv6 traffic is encapsulated in UDP packets on port 51820 and routed out through the Azure VM via NAT masquerading.



```
                        |  Outbound Internet Traffic
                        v
+-----------------------------------------+
|  Public Internet / Customer Website     |
+-----------------------------------------+
```

Why East US?

This deployment uses an Azure East US Virtual Machine to simulate a geographically distant VPN exit point for users located in India. This enables performance testing for use cases where internet traffic must traverse an international VPN before reaching public websites or hosted applications.
This setup helps quantify:

- Added network latency
- Throughput reduction
- Website load-time impact
- Overall user experience through the VPN

Prerequisites

- **Cloud Subscription:** An active Microsoft Azure account with permissions to create Virtual Machines, Public IPs, and Network Security Group (NSG) rules.
- **Client Device:** A Windows 10 or Windows 11 laptop with local Administrator rights to install the WireGuard virtual network adapter.
- **Command-Line Tools:** Windows PowerShell or Windows Terminal with built-in OpenSSH client (ssh) and curl.exe.

2. Cloud Infrastructure Provisioning (Azure)

Phase 1: Create the Azure Virtual Machine

1. Sign in to the **Azure Portal**, search for **Virtual machines**, and select **Create** → **Azure virtual machine**.
2. Configure the instance using the exact specifications below:

Parameter	Required Value	Operational Notes
Region	East US	Target exit region for US East routing tests.
Image	Ubuntu Server 22.04 LTS	Includes native WireGuard kernel module support.
VM Size	Standard_B1ms or Standard_B2s	B1ms suffices for light testing; B2s recommended for higher throughput.
Authentication	SSH public key or Password	Default username: azureuser (SSH key strongly recommended).
Public IP	Enabled (Static SKU recommended)	Prevents the VPN endpoint IP from changing after a VM reboot.

3. Click **Review + create**, then **Create**. Once deployment finishes, open the VM **Overview** page and record the assigned **Public IP address**.

Phase 2: Configure Network Security Group (NSG) Inbound Rules

Open the VM blade, navigate to **Networking** → **Network settings**, and ensure the following two inbound port rules are configured and saved:

Rule Name	Source	Destination Port	Protocol	Action
Allow-SSH	My IP (or trusted IPs)	22	TCP	Allow

Rule Name	Source	Destination Port	Protocol	Action
Allow-WireGuard	Any	51820	**UDP**	Allow

> **Critical Protocol Note:** WireGuard communicates exclusively over **UDP**. Selecting TCP for port 51820 will silently block all VPN handshake packets.

3. WireGuard Server Setup (Ubuntu VM)

Phase 3: Connect to the Azure VM via SSH

From PowerShell on your local Windows laptop, establish an SSH session to the Azure VM:

```
ssh azureuser@<SERVER_PUBLIC_IP>
```

Phase 4: Install WireGuard & Firewall Utilities

Update package repositories and install WireGuard along with iptables-persistent:

```
sudo apt update && sudo apt upgrade -y
sudo apt install wireguard iptables-persistent -y
wg --version
```

Phase 5: Generate Server Cryptographic Keys

Restrict file creation permissions (umask 077) so private keys are never world-readable, then generate the server keypair:

```
sudo mkdir -p /etc/wireguard
cd /etc/wireguard
umask 077
wg genkey | sudo tee /etc/wireguard/privatekey | wg pubkey | sudo tee
/etc/wireguard/publickey
```

Display both keys and copy them to a secure scratchpad for upcoming steps:

```
echo "SERVER PRIVATE KEY:" && sudo cat /etc/wireguard/privatekey
```

```
echo "SERVER PUBLIC KEY:" && sudo cat /etc/wireguard/publickey
```

Phase 6 & Phase 7: Enable Kernel IP Forwarding & Identify Network Interface

Enable IPv4 packet forwarding in the Linux kernel so traffic arriving on wg0 can be routed out to the internet:

```
echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
sysctl net.ipv4.ip_forward
```

Confirm the output reads net.ipv4.ip_forward = 1. Next, verify the primary outbound network interface name:

```
ip route | grep default
```

On standard Azure Ubuntu VMs, this returns default via 10.0.0.1 dev eth0 .... If your interface name differs from eth0 (such as ens5), substitute it in the NAT rules below.

Phase 8: Create WireGuard Server Configuration with Integrated NAT

Create /etc/wireguard/wg0.conf. Embedding PostUp and PostDown rules directly in the interface definition automatically applies NAT masquerading whenever the tunnel starts and cleans up rules cleanly on stop:

```
sudo nano /etc/wireguard/wg0.conf
```

Paste the following block, replacing <SERVER_PRIVATE_KEY> with the actual private key from Phase 5:

```
[Interface]
Address = 10.10.10.1/24
ListenPort = 51820
PrivateKey = <SERVER_PRIVATE_KEY>

PostUp = iptables -A FORWARD -i %i -j ACCEPT; iptables -A FORWARD -o %i -j
ACCEPT; iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i %i -j ACCEPT; iptables -D FORWARD -o %i
-j ACCEPT; iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE
```

Save and exit (Ctrl + O, Enter, Ctrl + X), then lock down file permissions:

```
sudo chmod 600 /etc/wireguard/wg0.conf
```

Phase 9: Enable & Start the WireGuard Service

```
sudo systemctl enable wg-quick@wg0
sudo systemctl start wg-quick@wg0
sudo wg show
```

Verify that interface: wg0 appears and reports listening port: 51820.

4. Windows Client Setup & Tunnel Activation

Phase 10 & Phase 11: Install Windows Client & Generate Client Keys

1. Download and install the official **WireGuard for Windows** client from the official WireGuard installation page.
2. Launch the WireGuard application.
3. Click the arrow next to **Add Tunnel** (bottom-left) and select **Add empty tunnel...**
4. WireGuard automatically generates a PrivateKey in the configuration editor and displays your **Public key** at the top of the dialog box.
5. Copy the **Client Public Key** and leave the editor window open.

Phase 12: Register Client Peer on the Azure Server

Return to your SSH terminal on the Azure VM and edit /etc/wireguard/wg0.conf:

```
sudo nano /etc/wireguard/wg0.conf
```

Append the client [Peer] block at the bottom of the file:

```
[Peer]
PublicKey = <CLIENT_PUBLIC_KEY>
AllowedIPs = 10.10.10.2/32
```

Save, exit, and restart the WireGuard service:

```
sudo systemctl restart wg-quick@wg0
sudo wg show
```

Scaling Note: To add additional client devices later, generate a separate keypair on each device and append new [Peer] blocks using sequential tunnel IPs (10.10.10.3/32, 10.10.10.4/32, etc.).

Phase 13 & Phase 14: Configure Client Tunnel & Activate

1. In the Windows WireGuard **Add empty tunnel** window, set the tunnel **Name** to Azure-EastUS-VPN.
2. Preserve your auto-generated PrivateKey and populate the full configuration below:

```
[Interface]
PrivateKey = <KEEP_AUTO_GENERATED_CLIENT_PRIVATE_KEY>
Address = 10.10.10.2/24
DNS = 8.8.8.8, 1.1.1.1

[Peer]
PublicKey = <SERVER_PUBLIC_KEY>
Endpoint = <SERVER_PUBLIC_IP>:51820
AllowedIPs = 0.0.0.0/0, ::/0
PersistentKeepalive = 25
```

3. Click **Save**, then click **Activate**.

Client Directive	Purpose
AllowedIPs = 0.0.0.0/0, ::/0	Enforces a full tunnel for all IPv4 and IPv6 traffic, preventing IPv6 leaks.
DNS = 8.8.8.8, 1.1.1.1	Routes DNS queries inside the tunnel to prevent local ISP DNS leaks.
PersistentKeepalive = 25	Pings every 25 seconds to maintain NAT translation state across home/office routers.

5. Connectivity Verification, Security & Troubleshooting

Phase 15: Verify Tunnel Handshake on Azure Server

Run sudo wg show on the Azure VM to confirm an active cryptographic handshake and bidirectional byte counters:

```
peer: <CLIENT_PUBLIC_KEY>
  endpoint: <CLIENT_ISP_PUBLIC_IP>:54321
  allowed ips: 10.10.10.2/32
  latest handshake: 15 seconds ago
  transfer: 45.20 KiB received, 112.40 KiB sent
```

Phase 16 & Phase 17: Verify Internal Routing & Egress Public IP on Windows

Open PowerShell on the Windows laptop and execute both checks:

```
# 1. Confirm internal tunnel reachability to server gateway
ping 10.10.10.1

# 2. Confirm outbound public IP matches Azure VM Public IP (not local ISP IP)
curl.exe ifconfig.me
```

Security Recommendations

- Use SSH key authentication instead of password authentication whenever possible.
- Restrict inbound SSH (TCP/22) access to trusted IP addresses.
- Never share or publish WireGuard private keys.
- Store configuration backups securely.
- Apply operating system updates regularly:

```
sudo apt update && sudo apt upgrade -y
```

- Remove unused WireGuard peers.
- Rotate VPN keys periodically in long-term deployments.
- Enable Azure monitoring and alerting for production environments.

Troubleshooting Matrix

Symptom	Likely Cause	Resolution
No latest handshake in sudo wg	UDP 51820 blocked or public keys swapped.	Confirm NSG allows **UDP 51820**. Verify Server Public Key is in Client config and Client Public Key is in Server config.
ping 10.10.10.1 works, but no	IP forwarding disabled or	Verify sysctl

Symptom	Likely Cause	Resolution
internet access	wrong NAT interface name.	net.ipv4.ip_forward equals 1 and confirm eth0 matches ip route | grep default.
Pages hang or load partially over VPN	Packet fragmentation due to MTU overhead.	Add MTU = 1380 under [Interface] on both server and client, then restart the tunnel.

6. Latency, Bandwidth & Load-Time Benchmarking

Run every test twice on the Windows laptop: first with the WireGuard tunnel **Deactivated (Direct Internet Baseline)**, and second with the tunnel **Activated (Via Azure East US)**.

Phase 18: Latency & Packet Loss Test

```
ping -n 20 google.com
ping -n 20 <CUSTOMER_WEBSITE_DOMAIN>
```

Record **Minimum (ms)**, **Maximum (ms)**, **Average (ms)**, and **Packet Loss (%)**.

Phase 19: Internet Bandwidth Test

This test measures end-user internet performance while connected through the VPN.

1. With the VPN **Off**, open **Speedtest by Ookla** in a browser and record **Download (Mbps)**, **Upload (Mbps)**, and **Ping (ms)**.
2. Activate the WireGuard tunnel, refresh the page, re-run the test, and record the VPN metrics.

Phase 20: Website HTTP/TLS Response Time Test

```
curl.exe -o NUL -s -w "DNS: %{time_namelookup}s | Connect:
%{time_connect}s | TLS: %{time_appconnect}s | Total Time:
%{time_total}s\n" https://<CUSTOMER_WEBSITE_DOMAIN>
```

Customer Performance Comparison Report Template

Performance Metric	Direct Internet (India ISP)	Via WireGuard (Azure East US)	Engineering Interpretation
Public Exit IP	Local ISP IP	Azure East US VM IP	Full traffic egress verified in US East.
Average Ping (RTT)	18 ms	220 ms	Expected transoceanic propagation delay (India ↔ US East).
Download Bandwidth	150 Mbps	95 Mbps	High throughput retained despite higher RTT windowing.
Upload Bandwidth	40 Mbps	30 Mbps	Minimal overhead from ChaCha20-Poly1305 encryption.
Packet Loss	0%	0%	Clean, stable UDP transport without drops.

7. End-to-End Deployment Sign-Off Checklist

Stage	Verification Item	Status
Azure VM	Ubuntu 22.04 LTS VM deployed in East US with	☐

Stage	Verification Item	Status
	active Public IP	
Azure NSG	Inbound firewall allows TCP 22 (restricted to trusted IPs) and **UDP 51820**	☐
WireGuard Server	WireGuard installed, keys generated (chmod 600), and /etc/wireguard/wg0.conf created	☐
Server Routing	net.ipv4.ip_forward=1 enabled, NAT masquerading active, and wg-quick@wg0 running	☐
Windows Client	Client installed, tunnel configured with 0.0.0.0/0, ::/0, peer added to server, tunnel activated	☐
Connectivity	sudo wg shows recent handshake, ping 10.10.10.1 succeeds, public IP matches Azure VM	☐
Benchmarking	Baseline vs. VPN latency, speedtest, and load-time metrics documented in report	☐